

Patch Prove

How the repair loop works, stage by stage.

Copies a repo. Asks DeepSeek for the smallest text edits. Applies them on the copy. Keeps a patch only if your test command passes. The original files stay as they are unless you pass `--write`.

INSIDE

- The broken fixture and the failing test, captured live.
- A disposable sandbox. Edits never start on `--repo`.
- Propose, apply, retest, reset. Then the original, still broken.
- Path escape rejected. `--write` is the only write-back.

```
$ bun run repair -- --repo fixtures/broken-math --test bun test
```

```
status: passed
attempt 1 tests-failed
  src/add.js
    - return a - b;
    + return a * b;
attempt 2 pass
  src/add.js
    - return a - b;
    + return a + b;
```

TESTS PASS
PATCH KEPT

Screenshots were captured from the real Bun modules on this machine.

The model was stubbed (no `DEEPSEEK_API_KEY`). The sandbox, patcher, and tests are live.

WHAT THIS IS

A loop that will not touch your tree until a patch has already passed.

Patch Prove is a local Bun program. There is no GitHub integration, no HTTP server, and no queue. You point it at a repo and a test command. It copies the tree, asks DeepSeek for the smallest search-and-replace edits, applies those edits on the copy, and runs your command. A patch is reported only if that command exits 0. The original files are left alone unless you pass `--write`.

This walkthrough uses the bundled fixture `fixtures/broken-math`: `add(2, 3)` is implemented as subtraction, and the test expects 5. Every screenshot after the live CLI check was produced by calling the same functions the CLI uses (`createSandbox`, `runTests`, `applyEdits`, `repair`). The proposer was stubbed because this machine has no `DEEPSEEK_API_KEY`.

The files

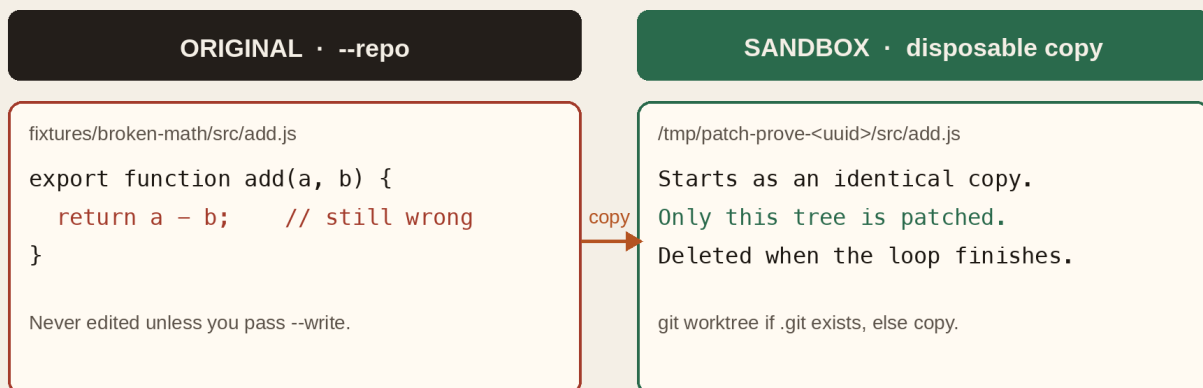
```

$ find . -type f | grep -v .git | sort
patchProve/
├── .env.example
├── fixtures/broken-math/src/add.js          # deliberately wrong
├── fixtures/broken-math/src/add.test.js     # expects add(2,3) === 5
├── src/cli.ts                             # flags, print, optional --write
├── src/repair.ts                           # the loop
├── src/sandbox.ts                         # copy / worktree / reset / destroy
├── src/patch.ts                           # safe {path,old,new}
├── src/propose.ts                         # pull JSON out of messy model text
├── src/llm.ts                             # DeepSeek chat/completions
├── src/testRunner.ts                      # Bun.spawn, no shell
└── tests/repair.test.ts

```

Repository layout captured from the working tree. The entire product is `src/repair.ts` plus six small helpers.

The two trees



Everything mutable lives on the right. The left-hand tree is read, then ignored, until an optional `--write` after a pass.

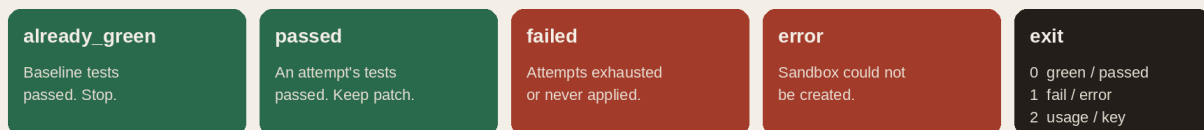
Inside the sandbox



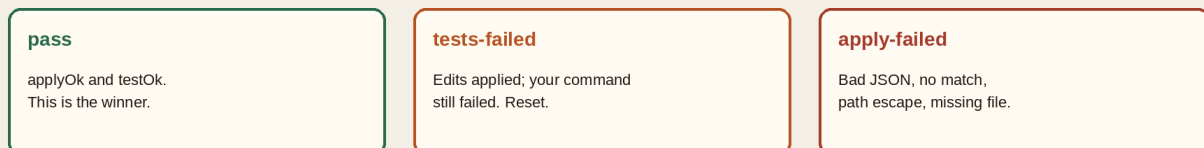
One pass through `repair()`. A failed attempt resets the copy and tries again, up to three times.

OUTCOMES

Four statuses, three attempt marks, one exit code.



Attempt marks printed by the CLI



Default is 3 attempts. Each failed attempt calls `resetSandbox()` so the next proposal starts from the original files again, with the latest test output.

Edits are { path, old, new }. The model never chooses `--test`. `--write` is opt-in.

Status values and CLI attempt marks. `already_green` is the quiet success: your command already passed, so the model is never called.

Flags

Flag	Meaning
--repo	Tree to copy. Required.
--test	Your command, argv only, no shell. Required. The model never chooses this.
--goal	Passed to the model. Default: make tests pass.
--write	After a pass, apply the winning edits to --repo. Off by default.
--max-attempts	How many propose/apply/test cycles. Default 3.
DEEPSEEK_API_KEY	Required by the live CLI. Optional base URL and model in .env.

Modules

File	Job
src/cli.ts	Parse flags, call repair(), print attempts, optional --write.
src/repair.ts	Sandbox, baseline test, attempt loop, return a result object.
src/sandbox.ts	git worktree if .git exists, else a filtered copy. Reset and destroy.
src/testRunner.ts	Bun.spawn the caller's argv. 30s default timeout. No shell.
src/llm.ts	DeepSeek chat/completions. Temperature 0.2. JSON-only system prompt.
src/propose.ts	First {...} in messy model text → { rationale, edits[] }.
src/patch.ts	Safe relative paths. Exact one match. Roll back the attempt on any error.

THE STAGES

A failing fixture, a copy, a patch, a proof.

Each screenshot is one moment in the loop. Captions under the terminals say exactly what was invoked. Read them in order: the original never changes until the last optional step.

01 The fixture is deliberately wrong

```
Stage 1 – the fixture is deliberately wrong · patchProve

$ cat fixtures/broken-math/src/add.js
export function add(a, b) {
  return a - b;
}

$ cat fixtures/broken-math/src/add.test.js
import { expect, test } from "bun:test";
import { add } from "./add.js";

test("adds two numbers", () => {
  expect(add(2, 3)).toBe(5);
});

# add(2, 3) currently returns -1. Tests are the source of truth.
```

This is the whole bug. add subtracts. The test is the source of truth and is not supposed to change unless the goal says so.

02 Your test command fails

```
Stage 2 – your test command fails · fixtures/broken-math

$ bun test
bun test v1.3.14 (0d9b296a)

src/add.test.js:
1 | import { expect, test } from "bun:test";
2 | import { add } from "./add.js";
3 |
4 | test("adds two numbers", () => {
5 |   expect(add(2, 3)).toBe(5);
  ^
error: expect(received).toBe(expected)

Expected: 5
Received: -1

at <anonymous>
(/home/sathwik/Dev/Projects/patchProve/fixtures/broken-math/src/add.test.js:5:21)
(fail) adds two numbers [0.12ms]

0 pass
1 fail
1 expect() calls
Ran 1 test across 1 file. [9.00ms]

# exit 1 – Patch Prove has not started yet.
# You still own the test command. The model never picks it.
```

Captured with runTests({ cwd: fixture, command: "bun test" }). Expected 5, received -1. Patch Prove has not started. You still own the command.

03 The live CLI will not start without a key

```
● ● ● The live CLI refuses to start without a key · patchProve

$ bun run repair -- --repo fixtures/broken-math --test bun test --goal "make tests pass"
set DEEPSEEK_API_KEY in .env

$ bun run repair --
usage: bun run repair -- --repo <path> --test <cmd> [--goal text] [--write]

# exit 2 in both cases. Nothing was copied.
```

This is the real `src/cli.ts` on this machine. Missing flags or a missing `DEEPSEEK_API_KEY` exit 2 and copy nothing. Every later stage uses `repair()` directly with a stub proposer.

04 A disposable copy is created

```
● ● ● Stage 4 – copy the repo into a disposable sandbox · sandbox

# createSandbox(repoPath)
mode:      copy
source:    /home/sathwik/Dev/Projects/patchProve/fixtures/broken-math
sandbox:    /tmp/patch-prove-733d119e-3431-4c73-bd3c-b0bad8a5a2fd

# no .git here, so the sandbox is a filtered copy (not a git worktree).
# node_modules / .git / dist are skipped; node_modules is symlinked if present.

$ cat src/add.js # inside the sandbox
export function add(a, b) {
  return a - b;
}

# The original and the copy start identical. Only the copy is edited.
```

The fixture has no `.git`, so the sandbox is a filtered copy under `/tmp/patch-prove-<uuid>`. A git checkout would have used `git worktree add --detach` instead. `node_modules`, `.git`, and `dist` are skipped; an existing `node_modules` is symlinked in.

05

Baseline tests run on the copy

```
Stage 5 - run your tests on the copy first · sandbox

# If this already passes, repair() returns already_green
# and never calls the model.

$ bun test # cwd = sandbox
bun test v1.3.14 (0d9b296a)

src/add.test.js:
1 | import { expect, test } from "bun:test";
2 | import { add } from "./add.js";
3 |
4 | test("adds two numbers", () => {
5 |   expect(add(2, 3)).toBe(5);
   ^
error: expect(received).toBe(expected)

Expected: 5
Received: -1

at <anonymous>
(/tmp/patch-prove-733d119e-3431-4c73-bd3c-b0bad8a5a2fd/src/add.test.js:5:21)
(fail) adds two numbers [0.28ms]

0 pass

status so far: red (ok=false)
# lastOutput is kept and sent to the model on the first attempt.
```

Same command, sandbox cwd. The stack path is the temp directory, not the original repo. Because this is red, the loop continues and the raw output is kept for the model.

06

A small file pack is gathered

```
Stage 6 - gather a small file pack for the model · sandbox

# listFiles() scans **/*.{js,ts,json}, skips node_modules/dist,
# drops binaries, keeps the first 200, then ranks names mentioned
# in the test output first, and sends at most 20 files (4 KB each).

files sent on this fixture:
  src/add.test.js    ← mentioned in failing output (ranked first)
  src/add.js
  package.json

# The model sees: goal, attempt number, raw test output, file pack.
# It does not see the test command, and must not invent one.
```

listFiles scans **/*.{js,ts,json}, drops binaries, keeps the first 200, ranks names that appear in the test output, and sends at most 20 files truncated to 4 KB. The test file is first because the failure names it.

07

The model may only return tiny replacements

```

● ● ● Stage 7 – the model returns tiny search-and-replace edits

# proposeWithLlm() asks DeepSeek for JSON only.
# parseEdits() pulls the first {...} out of messy model prose.

system: You fix a failing repo with the smallest
        search-and-replace edits. Return ONLY JSON.

{
  "rationale": "The implementation subtracts; the test expects addition.",
  "edits": [
    {
      "path": "src/add.js",
      "old": "return a - b;",
      "new": "return a + b;"
    }
  ]
}

parsed rationale: The implementation subtracts; the test expects addition.
parsed edits:      1 (src/add.js)
# Rules: existing files only · old must match exactly once
#         do not change tests unless the goal says to
#         no new files, no deletes, no unified diffs

```

Live, this is DeepSeek deepseek-v4-flash with temperature 0.2. Here the JSON is the shape parseEdits actually accepts — including the leading prose, which is stripped. Edits are { path, old, new }, not unified diffs.

08

Edits land on the copy only

```

● ● ● Stage 8 – apply the edits on the copy, never on --repo

$ applyEdits(sandbox.path, edits)
applyOk: true

sandbox/src/add.js
export function add(a, b) {
  return a + b; ← changed
}

fixtures/broken-math/src/add.js (the original)
export function add(a, b) {
  return a - b;
}

# If any edit fails (missing file, 0 or 2+ matches, path escape,
# empty old text, binary), every file this call already changed
# is restored and the attempt is marked apply-failed.

```

Captured after a real applyEdits(sandbox.path, edits). The sandbox now adds; the fixture still subtracts. If any edit fails, every file this call already changed is restored.

09

The same test command is run again

```
● ● ● Stage 9 – run the same test command on the patched copy · sandbox

$ bun test # still cwd = sandbox, same argv as --test
bun test v1.3.14 (0d9b296a)

src/add.test.js:
(pass) adds two numbers [0.11ms]

1 pass
0 fail
1 expect() calls
Ran 1 test across 1 file. [9.00ms]

testOk: true → this attempt is the winner
```

Same argv, sandbox cwd. Exit 0 means this attempt is the winner. `repair()` returns status: passed and winningEdits.

10

The sandbox is destroyed; the original is still broken

```
● ● ● Stage 10 – destroy the sandbox; the original is still broken · patchProve

# destroySandbox() deletes the temp copy (or git worktree remove).

$ cat fixtures/broken-math/src/add.js
export function add(a, b) {
  return a - b;
}

source tree unchanged – --write was not passed
# That is the whole point: you only see a patch that already passed
# your tests, and nothing is written back unless you ask.
```

This is the product promise. A passing patch was proven on a copy. Nothing was written back because `--write` was not passed. The project's own tests assert this after every `repair()` call.

11

A bad proposal is discarded and the copy is reset

```

● ● ● Stage 11 – a bad proposal is discarded; the sandbox is reset

# Same repair() the CLI uses. First proposal multiplies (still wrong).
# Tests fail → resetSandbox() restores the copy → attempt 2 adds.

status: passed
attempt 1 tests-failed
  Maybe multiplication will make 2 and 3 into 5.
  src/add.js
  - return a - b;
  + return a * b;
attempt 2 pass
  The implementation subtracts; the test expects addition.
  src/add.js
  - return a - b;
  + return a + b;

winningEdits: {"path":"src/add.js","old":"return a - b;","new":"return a + b;"}
# After a failed attempt the next propose() sees the latest test output.
# Default maxAttempts is 3. Exhausted attempts → status failed.

```

A real `repair()` run. Attempt 1 multiplies (still `add(2,3) !== 5`), so the sandbox is reset. Attempt 2 adds and the tests pass. Default budget is three attempts.

12

A path that leaves the sandbox is rejected

```

● ● ● Stage 12 – path escape is rejected; source stays put

# resolveSafePath() refuses absolute paths and anything that
# would walk out of the sandbox with ..

status:      failed
applyOk:     false
error:       path escapes sandbox: ../secret

$ cat fixtures/broken-math/src/add.js
export function add(a, b) {
  return a - b;
}
fixture still contains  return a - b;

```

`resolveSafePath` refused `../secret`. The attempt is `apply-failed`, the fixture still subtracts.

13 --write is the only write-back

```

● ● ● Stage 13 - --write is the only way the original changes

$ bun run repair -- --repo fixtures/broken-math --test bun test --write

# After status === passed, CLI calls applyEdits(repo, winningEdits).
# If status is anything else it prints 'not writing' and exits 1.

if (write) {
  if (result.status !== "passed") {
    console.error("not writing: status is " + result.status);
    process.exit(1);
  }
  await applyEdits(repo, result.winningEdits);
}

# This walkthrough did not pass --write. The fixture is still broken,
# which is what the project's own tests assert after every repair().

```

The CLI applies winningEdits to --repo only after status === "passed". Any other status prints not writing and exits 1.

14 What you see when the CLI finishes

```

● ● ● What the CLI prints when it finishes

# cli.ts prints this shape - one block per attempt - then last test output.

status: passed
attempt 1 tests-failed
  Maybe multiplication will make 2 and 3 into 5.
  src/add.js
  - return a - b;
  + return a * b;
attempt 2 pass
  The implementation subtracts; the test expects addition.
  src/add.js
  - return a - b;
  + return a + b;

last test output:
(pass) adds two numbers [0.08ms]
1 pass
Ran 1 test across 1 file. [11.00ms]

# exit 0 on already_green | passed
# exit 1 on failed | error
# exit 2 on bad flags or missing DEEPSEEK_API_KEY

```

This is the printout src/cli.ts writes: a status, one block per attempt, then the last test output. Exit 0 on already_green or passed.

15

The contract, tested

A terminal window with a dark background and light green text. The title bar shows three colored circles (red, yellow, green) followed by the text "The contract is locked in by seven tests" and a separator followed by "patchProve". The terminal content shows a command prompt "\$" followed by the command "bun test tests". Below this, it says "bun test v1.3.14 (0d9b296a)". Then, it lists the test file "tests/repair.test.ts:" followed by seven test results, all marked as "(pass)". The tests are: "correct edit passes and leaves the fixture broken", "bad then good edit passes on the second try", "always-bad edits fail after 3 tries", "already-green runs propose zero times", "propose gets the file named in the failing test", "parseEdits reads JSON wrapped in prose", and "path escape is rejected and source is untouched". Below the tests, it shows "7 pass", "0 fail", "18 expect() calls", and "Ran 7 tests across 1 file. [240.00ms]".

```
● ● ● The contract is locked in by seven tests · patchProve

$ bun test tests
bun test v1.3.14 (0d9b296a)

tests/repair.test.ts:
(pass) correct edit passes and leaves the fixture broken
(pass) bad then good edit passes on the second try
(pass) always-bad edits fail after 3 tries
(pass) already-green runs propose zero times
(pass) propose gets the file named in the failing test
(pass) parseEdits reads JSON wrapped in prose
(pass) path escape is rejected and source is untouched

 7 pass
 0 fail
18 expect() calls
Ran 7 tests across 1 file. [240.00ms]
```

Captured with `bun test tests` after this walkthrough. The fixture is still broken. That is required.

LOCKED BEHAVIOUR

What the loop will not do.

- The model never chooses the test command.
- Edits are { path, old, new }. No unified diffs, no new files, no deletes.
- old must match exactly once. Zero matches or two-plus matches fail the attempt.
- Paths must be relative and must stay inside the sandbox.
- Binary files are refused. Empty old text is refused.
- Default three attempts. The sandbox is reset after every failed attempt.
- The source tree is unchanged unless the caller passes `--write` after a pass.
- No GitHub, no queue, no database, no HTTP server.
- Tests are not edited unless the goal says to.

How to run it yourself

`cp .env.example .env` and set `DEEPSEEK_API_KEY`. Then:

```
bun test tests
```

```
bun run repair -- --repo fixtures/broken-math --test bun test --goal "make tests pass"
```

Add `--write` only if you want the winning edits applied to `--repo` after a pass.

This document was generated from a live run of the repository on 17 August 2026. The fixture was left broken on purpose.